[수행과제**]** 컴퓨터 비전**(Computer Vision) AI** 웹 서비스 제작 및 기획서 작성

1. 과제 개요 및 목표

일상생활이나 산업 현장의 고충(Pain Point)을 컴퓨터 비전(Computer Vision) AI 기술로 해결하는 웹앱 서비스를 제작하는 기획서를 작성

단순히 "AI가 사진을 인식해 준다"는 기능적 나열을 넘어,

- 왜 비전 AI가 필요한지
- 학습 데이터는 어떻게 확보할 것인지
- 어떻게 개발할 것인지

를 논리적으로 설계하여 실제 MVP(최소기능제품) 개발이 가능한 수준의 기획서를 완성하는 것이 목표

2. 기획 주제 선택 가이드 **(**택 **1** 또는 자유 주제**)**

주제 선정이 어려운 경우 아래 예시 중 하나를 선택하여 구체화하거나, 본인의 관심 분야를 자유롭게 선정

분야 **A.** 헬스케어 **&** 피트니스: 웹캠 활용 홈트레이닝 실시간 자세 교정 및 운동 횟수 카운팅 웹앱 (Pose Estimation 활용)
분야 **B.** 환경 **&** 라이프스타일: 스마트폰 카메라로 촬영한 재활용 쓰레기의 재질 및 오염도를 분석해 올바른 분리배출 방법을 안내하는 웹 서비스 (Object Detection & Classification 활용)
분야 **C.** 리테일 **&** 커머스: 냉장고 내부 사진을 찍으면 식재료를 자동 인식하여 유통기한을 관리하고 맞춤형 레시피를 추천하는 웹앱 (Multi-label Classification / OCR 활용)
분야 **D.** 안전 **&** 산업 현장: 건설/물류 현장 작업자의 안전모·안전조끼 착용 여부를 웹캠으로 실시간 감지하여 미착용 시 경고 알림을 보내는 관제 대시보드 (Real-time Object Detection 활용)

3. 기획서 포함 항목 **(**보편적인 **AI** 공모전의 **4**대 핵심 영역**)**

기획서에는 아래 4가지 섹션이 포함되어야 하며, 각 항목의 핵심 질문에 대한 해답이 제시되어야 함.

① 서비스 개요 및 문제 정의 **(Problem Definition)**

- 타겟 페르소나 및 페인 포인트: 누가, 어떤 상황에서, 어떤 불편함을 겪고 있는가?
- 기술 채택의 당위성: 기존 텍스트 입력이나 단순 규칙(Rule-based) 웹 서비스가 아닌, 컴퓨터 비전 AI(객체 탐지, 이미지 분할, 자세 추정, OCR 등)를 도입하면 좋은 이유는 무엇인가?
- 핵심 가치 제안 (UVP): 이 웹 서비스를 통해 사용자가 얻는 정량적·정성적 효용은 무엇인가?

② **AI** 모델 및 데이터 전략 **(AI & Data Strategy)**

- 적용 기술 및 모델 아키텍처: 문제 해결에 적합한 비전 태스크와 오픈소스/상용 모델(예: YOLO, MediaPipe, SAM, Vision API 등) 선정 이유.
- 데이터 수집 및 가공(Annotation) 전략:
  초기 모델 학습을 위한 데이터 확보 방안 (공공 데이터셋 활용, 자체 크롤링, 사용자 참여형 수집 등).
  라벨링 방식(Bounding Box, Polygon, Keypoint 등) 및 데이터 품질 관리 기준.
- 성능 목표 및 예외 상황(Edge Case) UX 대응:
  목표 성능 지표 설정 (예: 정밀도/재현율 85% 이상, 웹 환경 추론 속도 1.5초 이내 등).
  AI 한계점 보완책: 역광/저조도 환경, 피사체 가려짐(Occlusion), 모델의 오인식(False Positive) 발생 시 사용자 이탈을 막기 위한 UI/UX적 해결책 제시 (예: 촬영 가이드라인 오버레이 제공, 신뢰도 점수 표시, 오인식 결과에 대한 사용자 수동 수정 인터페이스 등).

③ 사용자 여정 및 화면 설계 **(UX/UI & Wireframe)**

- 핵심 유저 플로우 (User Flow): 웹 접속 → 권한 허용(카메라/갤러리) → 이미지 입력 → AI 분석 대기 → 결과 확인 및 후속 액션까지의 흐름도.
- 주요 화면 와이어프레임:
  입력 화면: 드래그 앤 드롭 업로드 또는 실시간 웹캠 스트리밍 UI.
  분석 대기 화면: 모델 추론 시간 동안 신뢰감을 주는 로딩 인터랙션(스켈레톤 UI, 분석 단계 안내 등)
  결과 시각화 화면: 원본 이미지 위 바운딩 박스/히트맵 오버레이 및 직관적인 분석 리포트 대시보드.

④ 비즈니스 모델**(**선택**)** 및 실행 로드맵 **(Business & Roadmap)**

- 수익화 모델 (BM): B2C 구독형, B2B SaaS/API 과금형, 제휴 커머스 연계 등 지속 가능한 수익 구조.
- 단계별 추진 일정 (Roadmap): 데이터 수집 및 전처리 → AI 모델 PoC(개념검증) → 웹 프론트엔드/백엔드 연동(MVP) → 베타 테스트까지의 마일스톤.

작성 가이드

1. 프로젝트 개요 및 목적 (Core Concept)
AI가 '어떤 목적'을 위해 '누가' 쓰는 앱을 만드는지 전체적인 맥락을 이해할 수 있어야 함.
서비스 주제: 예) 실시간 웹캠 기반 자세 교정 서비스, 업로드한 제품 이미지 결함 검출기 등
핵심기능(MVP): 사용자가 앱에 접속해 최종적으로 얻게되는 결과물 (예: 이미지 업로드 → AI 객체 탐지 → 결과 바운딩박스 및 통계 표출)

2. 컴퓨터비전 처리 방식 및 아키텍처 (AI Architecture)
컴퓨터비전 기능이 브라우저에서만 실행 될 것인지, 서버에 배포 후 실행 될 것인지, 또는 앱으로 만들어 배포 할 것인지에 따라 기술 스택과 코드가 완전히 달라짐. 이 부분을 명확히 지정해 주어야 함.

추론 모델 구동 환경 방식 선택:
프론트엔드 단독 (Client-Side): 브라우저에서 TensorFlow.js, ONNX Runtime Web, MediaPipe 등을 이용해 클라이언트의 GPU/CPU로 직접 모델을 돌리는 방식 (서버 비용 절감, 실시간 처리에 유리)
백엔드 처리 (Server-Side): 사용자가 이미지를 올리면 FastAPI, Flask 등의 서버로 보내 Python(PyTorch, OpenCV, Ultralytics YOLO 등)으로 처리 후 결과를 받아오는 방식 (무거운 모델 사용 가능)

3. 기술 스택 및 패키지 명시 (Tech Stack & Libraries)
사용하고 싶은 라이브러리나 프레임워크가 있다면 버전과 함께 명시해야 LLM모델이 호환되지 않는 코드를 작성하는 것을 막을 수 있음.
프론트엔드: React, Next.js, Vue 등 + 스타일링 도구 (Tailwind CSS 등)
백엔드 (서버 방식 선택 시): Python FastAPI 또는 Flask
컴퓨터비전 관련 핵심 패키지:예시: opencv-python, ultralytics (YOLOv8 등), mediapipe, torch, onnxruntime 등

4. 작성 및 제출 유의사항
시각화 권장: 텍스트 위주의 서술만 하지 말고, 유저 플로우, 아키텍처 개념도, 화면 와이어프레임 등을 활용
도구 활용: 문서작성은 가급적 Notion, 바이브코딩은 Codex, Claud, Cursor 등, 와이어프레임은 Figma 등 본인이 다루기 편한 도구를 자유롭게 사용
생성형 AI 활용 가이드: 자료 조사 및 초안 구조화 과정에서 ChatGPT, Gemini 등 생성형 AI를 보조 도구로 활용할 수 있으나, 서비스의 핵심 기획 의도와 화면 설계는 반드시 본인의 논리로 재구성해야 함.

제출 산출물
1. 기획서 1부 (PDF 변환, 원본 Notion/Figma/Docx/PPT 링크 첨부 가능)
2. 바이브 코딩 초기 Context 용 프롬프트 .md

예시1)

None

```
[프로젝트 개요]
- 주제: 웹캠 기반 실시간 객체(사람/사물) 탐지 및 카운팅 웹앱
- 목적: 사용자가 카메라를 켤 때 실시간으로 특정 객체를 감지하고 화면에 통계를 보여줌.

[아키텍처 및 기술 스택]
- 방식: 프론트엔드(Client-side) 처리 선호 (TensorFlow.js 또는 MediaPipe 활용, 혹은 가벼운 ONNX 모델 브라우저 구동)
- 프레임워크: React(Vite) + Tailwind CSS
- UI 구성: 좌측에는 웹캠 실시간 영상(Canvas 오버레이 포함), 우측에는 감지된 객체 리스트 및 실시간 카운트 대시보드

[핵심 요구사항]
1. 브라우저에서 웹캠 권한을 요청하고 스트림을 <video> 및 <canvas>에 연결할 것.
2. AI 모델이 프레임마다 객체를 탐지하여 Canvas 위에 바운딩 박스와 라벨을 실시간으로 그려줄 것.
3. 성능 저하를 방지하기 위해 프레임 스킵(Frame Skipping) 로직을 포함할 것.

이 구조를 바탕으로 초기 파일 구조와 기본 코드를 작성해 줘.
```

예시2) 역프롬프트로 만든 초기 프롬프트

None

```
# 프로젝트: VisionCount AI – 산업현장 부품 수량 자동 비전검사 웹앱

## [프로젝트 개요]
- 주제: AI 컴퓨터비전 기반 산업현장 부품 수량 자동 검사 웹앱
- 목적: 작업자가 촬영한 부품 이미지를 AI가 분석하여 부품별 수량을 자동 검출하고 기준 수량과 비교하여 OK/NG를 판정하는 시스템
- 대상: 제조업 생산라인, 부품 조립공정, 품질검사 현장
- 개발 목표: 로컬 PC에서 실행 가능한 웹 기반 비전검사 시스템 MVP 구현
- 개발 방식: Codex를 활용한 단계별 바이브 코딩

## [아키텍처 및 기술 스택]
**Frontend**
- React + Vite
- TypeScript
- Tailwind CSS
- 반응형 웹 UI

**Backend**
- Python
- FastAPI
- REST API

**Computer Vision**
- OpenCV
- Ultralytics YOLO 기반 객체 검출
- 향후 사용자 정의 부품 데이터셋 학습 및 커스텀 모델 적용
```

Database
- SQLite (MVP)
- 향후 PostgreSQL 확장 가능 구조

개발 환경
- 로컬 PC 실행
- Python 가상환경 사용
- 프론트엔드와 백엔드 분리
- 환경변수 기반 설정
- 무료 오픈소스 기술 우선 사용
- 상업적 사용 시 라이브러리 및 AI 모델 라이선스 확인

[핵심 요구사항]
1. 이미지 검사
- JPG, PNG 이미지 업로드
- 업로드 이미지 미리보기
- AI 객체 검출 실행
- 검출된 부품별 바운딩 박스 표시
- 객체별 번호 및 Confidence 표시
- 검출된 부품 총수량 자동 계산

2. 수량 판정
- 검사 품목 선택 및 설정
- 기준 수량 입력
- AI 검출 수량 표시
- 기준 수량과 검출 수량 비교
- 수량 일치 시 OK 판정
- 수량 불일치 시 NG 판정
- 부족 수량 및 초과 수량 표시
- 검출 불가 또는 신뢰도 부족 시 '검토 필요' 표시

3. 검사 결과
- 원본 이미지와 검출 결과 이미지 비교
- 기준 수량 / 검출 수량 / 차이 수량 표시
- OK / NG 판정 결과 표시
- 검사 일시 및 AI 처리 시간 표시
- 검사 결과와 이미지 저장

4. 검사 이력
- 검사 날짜 및 시간
- 검사 품목명
- 기준 수량 및 검출 수량
- OK / NG 판정 결과
- 검사 이미지
- 최근 검사 이력 조회

[UI/UX 구성]
산업현장에서 실제 사용할 수 있는 스마트팩토리 스타일의 전문적인 검사 대시보드를 구현한다.

디자인 방향
- 다크 모드 기반 산업용 UI
- 깔끔하고 현대적인 디자인
- 큰 글씨와 직관적인 버튼

- 터치스크린 작업 환경 고려
- 검사 결과의 높은 시인성 확보
- 다양한 화면 크기에 대응하는 반응형 레이아웃

메인 화면 레이아웃

상단:
- 시스템명: VisionCount AI
- 시스템 연결 및 동작 상태
- 현재 날짜와 시간

좌측 – 검사 설정:
- 검사 품목 선택
- 기준 수량 입력
- 이미지 업로드
- 검사 시작 버튼
- 검사 초기화 버튼

중앙 – 비전검사 화면:
- 원본 이미지 표시
- AI 검출 결과 이미지 표시
- 객체별 바운딩 박스
- 검출 객체 번호 및 신뢰도

우측 – 검사 결과:
- 기준 수량
- 실제 검출 수량
- 수량 차이
- OK / NG / 검토 필요 판정
- AI 처리 시간

하단 – 검사 이력:
- 최근 검사 목록
- 검사 일시
- 품목명
- 기준 수량 및 검출 수량
- 판정 결과

상태별 색상
- OK: 녹색
- NG: 빨간색
- 검토 필요: 노란색
- 검사 진행 중: 파란색

[AI 검출 설계 원칙]
1. AI 객체 검출 엔진과 수량 판정 로직을 분리한다.
2. 일반 사전학습 YOLO 모델이 산업용 특수 부품을 정확하게 인식한다고 가정하지 않는다.
3. 초기에는 실제 AI 검출 모드와 DEMO 모드를 명확하게 구분한다.
4. 실제 AI 모델이 인식하지 못하는 부품의 수량을 임의로 생성하지 않는다.
5. 향후 학습된 커스텀 YOLO 모델 가중치를 연결할 수 있도록 설계한다.
6. Confidence 임계값을 설정할 수 있도록 구조를 설계한다.
7. 중복 검출을 줄이기 위한 NMS 적용을 고려한다.
8. AI 추론 오류, 모델 미설치, 검출 불가 상태를 명확히 처리한다.
9. 신뢰할 수 없는 결과를 자동으로 OK 판정하지 않는다.

10. DEMO 모드에서는 실제 AI 검출 결과가 아님을 화면에 명확히 표시한다.

[개발 단계]
Phase 1 – 실행 가능한 MVP
- React + Vite 프로젝트 생성
- FastAPI 백엔드 구축
- 산업용 대시보드 UI 구현
- 이미지 업로드 및 미리보기
- 검사 REST API 구현
- DEMO 검사 모드 구현
- 기준 수량과 데모 검출 수량 비교
- OK/NG 판정 화면 구현
- 프론트엔드와 백엔드 연동

Phase 2 – AI 컴퓨터비전 연동
- Ultralytics YOLO 모델 연동
- 실제 객체 검출 API 구현
- 바운딩 박스 및 객체 번호 표시
- 부품 수량 자동 계산
- Confidence 및 NMS 설정
- 커스텀 YOLO 모델 적용 구조

Phase 3 – 검사 시스템 고도화
- 품목별 검사 설정 관리
- 기준 수량 관리
- SQLite 검사 이력 저장
- 원본 및 결과 이미지 저장
- 검사 이력 검색
- 검사 통계 대시보드

Phase 4 – 산업현장 확장
- USB 웹캠 실시간 검사
- 산업용 카메라 연동
- 실시간 객체 검출 및 카운팅
- PLC 연동용 API 설계
- 사용자 정의 부품 모델 적용
- 검사 결과 CSV 다운로드

[개발 지시]
너는 숙련된 풀스택 개발자이자 컴퓨터비전 AI 엔지니어의 관점에서 개발을 진행하라.

다음 개발 원칙을 준수한다.

1. 먼저 현재 작업 디렉터리와 개발 환경을 확인한다.
2. 기존 파일이 있을 경우 임의로 삭제하거나 덮어쓰지 않는다.
3. 전체 프로젝트 폴더 구조를 간략하게 설계한다.
4. 설명이나 코드 예시만 제시하지 말고 실제 프로젝트 파일을 생성한다.
5. 프론트엔드와 백엔드를 분리하여 구성한다.
6. 실제로 작동하는 기능부터 단계적으로 구현한다.
7. 프론트엔드와 백엔드 API 연결을 검증한다.
8. 사용자 입력 및 API 오류를 적절하게 처리한다.
9. 필요한 패키지와 실행 명령어를 README.md에 작성한다.
10. 주요 기능 구현 후 테스트와 빌드를 수행한다.
11. 오류가 발생하면 원인을 분석하고 수정한다.

12. 과도한 설계와 불필요한 의존성을 피한다.
13. 향후 AI 모델과 데이터베이스를 쉽게 확장할 수 있는 구조를 유지한다.
14. 구현되지 않은 기능을 실제 작동하는 것처럼 표시하지 않는다.
15. 기술적으로 판단이 필요한 사항은 합리적인 기본값을 선택하여 진행한다.

[첫 번째 작업: Phase 1 MVP 구현]
지금부터 Phase 1 개발을 시작하라.

구현 작업
1. 현재 개발 환경 및 프로젝트 구조 확인
2. React + Vite + TypeScript + Tailwind CSS 초기 설정
3. Python FastAPI 백엔드 구성
4. VisionCount AI 메인 대시보드 UI 구현
5. 검사 품목 및 기준 수량 입력 기능
6. 이미지 업로드 및 미리보기 구현
7. DEMO 검사 API 구현
8. 프론트엔드와 FastAPI 연결
9. 기준 수량과 데모 검출 수량 비교 및 OK/NG 판정
10. 검사 시작 및 초기화 버튼 구현
11. 로딩, 오류 및 검사 상태 표시
12. README.md 작성 및 실행 테스트

DEMO 모드 구현 조건
- 실제 YOLO 검출 기능은 Phase 2에서 구현한다.
- Phase 1에서는 데모 데이터로 전체 검사 흐름을 검증한다.
- 데모 검출 수량은 테스트 가능한 명시적 값으로 제공하며 실제 이미지에서 계산된 값처럼 표시하지 않는다.
- DEMO 모드에서는 화면에 'DEMO' 표시를 유지한다.
- 실제 AI 모델이 연결되지 않았다는 사실을 명확히 안내한다.

[완료 기준]
다음 조건을 모두 만족해야 Phase 1이 완료된 것으로 판단한다.

1. 로컬 브라우저에서 웹앱이 정상 실행된다.
2. 사용자가 JPG 또는 PNG 이미지를 업로드할 수 있다.
3. 업로드한 이미지가 화면에 표시된다.
4. 검사 품목과 기준 수량을 설정할 수 있다.
5. 검사 시작 버튼을 누르면 FastAPI 엔드포인트가 호출된다.
6. DEMO 검출 수량이 화면에 표시된다.
7. 기준 수량과 비교하여 OK/NG를 정확하게 판정한다.
8. 검사 초기화 버튼이 정상 작동한다.
9. DEMO 데이터와 실제 AI 검출 결과를 혼동하지 않도록 표시한다.
10. 프로젝트 실행 방법이 README.md에 정리되어 있다.
11. 가능한 테스트와 빌드가 성공하며, 실패 항목이 있다면 원인을 보고한다.

중요: 이번 작업은 Phase 1 MVP 구현에 집중한다. Phase 2~4 기능은 구현하지 말고 향후 개발 계획으로만 유지한다.

위 요구사항을 바탕으로 프로젝트 환경을 확인하고, 초기 파일 구조 설계부터 실제 코드 작성, API 연동, 테스트까지 진행하라.